DS HW 5

1. Use linked list for polynomial representation and perform addition and subtraction operations on two polynomials.

Program:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int coeff;
    int power;
    struct Node *next;
};

struct Node* createNode(int c, int p) {
    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->coeff = c;
    newNode->power = p;
    newNode->next = NULL;
    return newNode;
}

void insert(struct Node **head, int c, int p) {
    struct Node *newNode = createNode(c, p);
    struct Node *temp;

    if (*head == NULL) {
        *head = newNode;
        return;
    }

    temp = *head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
}

struct Node* add(struct Node *p1, struct Node *p2, int sign) {
    struct Node *result = NULL;

    while (p1 != NULL && p2 != NULL) {
        if (p1->power == p2->power) {
            insert(&result, p1->coeff + sign * p2->coeff, p1->power);
            p1 = p1->next;
            p2 = p2->next;
        }
        else if (p1->power > p2->power) {
            insert(&result, p1->coeff, p1->power);
            p1 = p1->next;
        }
        else {
            insert(&result, sign * p2->coeff, p2->power);
            p2 = p2->next;
        }
    }

    while (p1 != NULL) {
        insert(&result, p1->coeff, p1->power);
        p1 = p1->next;
    }
```

```
    while (p2 != NULL) {
        insert(&result, sign * p2->coeff, p2->power);
        p2 = p2->next;
    }

    return result;
}

void display(struct Node *p) {
    while (p != NULL) {
        printf("%dx^%d", p->coeff, p->power);
        if (p->next != NULL)
            printf(" + ");
        p = p->next;
    }
    printf("\n");
}

int main() {
    struct Node *p1 = NULL, *p2 = NULL;
    struct Node *sum, *diff;

    insert(&p1, 5, 2);
    insert(&p1, 4, 1);
    insert(&p1, 3, 0);

    insert(&p2, 3, 2);
    insert(&p2, 2, 1);
    insert(&p2, 1, 0);

    printf("Polynomial 1: ");
    display(p1);

    printf("Polynomial 2: ");
    display(p2);

    sum = add(p1, p2, 1);
    diff = add(p1, p2, -1);

    printf("Addition: ");
    display(sum);

    printf("Subtraction: ");
    display(diff);

    return 0;
}
```

Output:

```
Polynomial 1: 5x^2 + 4x^1 + 3x^0
Polynomial 2: 3x^2 + 2x^1 + 1x^0
Addition: 8x^2 + 6x^1 + 4x^0
Subtraction: 2x^2 + 2x^1 + 2x^0
```

2. Find the previous node of a given node in a circular linked list.

Program:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

int main() {
    struct Node *head = NULL, *newNode, *temp;
    int n, value, i;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        newNode = (struct Node*)malloc(sizeof(struct Node));

        printf("Enter data: ");
        scanf("%d", &newNode->data);

        if (head == NULL) {
            head = newNode;
            newNode->next = head;
        } else {
            temp = head;

            while (temp->next != head)
                temp = temp->next;

            temp->next = newNode;
            newNode->next = head;
        }
    }

    printf("Enter node value: ");
    scanf("%d", &value);

    temp = head;

    do {
        if (temp->next->data == value) {
            printf("Previous node of %d is %d\n",
                   value, temp->data);
            return 0;
        }
        temp = temp->next;
    } while (temp != head);

    printf("Node not found\n");

    return 0;
}
```

Output:

```
Enter number of nodes: 5
Enter data: 10
Enter data: 20
Enter data: 30
Enter data: 40
Enter data: 50
Enter node value: 30
```

```
Previous node of 30 is 20
```

3. Insert an element at a given position in a circular linked list.

Program:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

int main() {
    struct Node *head = NULL, *newNode, *temp;
    int n, i, pos, value;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        newNode = (struct Node*)malloc(sizeof(struct Node));

        printf("Enter data: ");
        scanf("%d", &newNode->data);

        if (head == NULL) {
            head = newNode;
            newNode->next = head;
        } else {
            temp = head;

            while (temp->next != head)
                temp = temp->next;

            temp->next = newNode;
            newNode->next = head;
        }
    }

    printf("Enter position: ");
    scanf("%d", &pos);

    printf("Enter value: ");
    scanf("%d", &value);

    newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;

    if (pos == 1) {
        temp = head;

        while (temp->next != head)
            temp = temp->next;

        newNode->next = head;
        temp->next = newNode;
        head = newNode;
    } else {
        temp = head;

        for (i = 1; i < pos - 1; i++)
            temp = temp->next;

        newNode->next = temp->next;
        temp->next = newNode;
    }
```

```
    printf("Circular Linked List: ");

    temp = head;

    do {
        printf("%d ", temp->data);
        temp = temp->next;
    } while (temp != head);

    return 0;
}
```

Output:

```
Enter number of nodes: 4
Enter data: 10
Enter data: 20
Enter data: 30
Enter data: 40
Enter position: 3
Enter value: 25
Circular Linked List: 10 20 25 30 40
```

4. Find the largest and second largest element in a queue.

Program:

```
#include <stdio.h>

int main() {
    int queue[100], n;
    int i, largest, second;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter queue elements:\n");

    for (i = 0; i < n; i++)
        scanf("%d", &queue[i]);

    largest = second = -99999;

    for (i = 0; i < n; i++) {
        if (queue[i] > largest) {
            second = largest;
            largest = queue[i];
        }
        else if (queue[i] > second && queue[i] != largest) {
            second = queue[i];
        }
    }

    printf("Largest element = %d\n", largest);
    printf("Second largest element = %d\n", second);

    return 0;
}
```

Output:

```
Enter number of elements: 5
Enter queue elements:
10 40 20 50 30
Largest element = 50
Second largest element = 40
```

5. Insert the 3rd element and check whether the 1st and 3rd elements are the same in a linked list.

Program:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

int main() {
    struct Node *head = NULL, *newNode, *temp;
    int n, i, value;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        newNode = (struct Node*)malloc(sizeof(struct Node));

        printf("Enter data: ");
        scanf("%d", &newNode->data);

        newNode->next = NULL;

        if (head == NULL)
            head = newNode;
        else {
            temp = head;
            while (temp->next != NULL)
                temp = temp->next;

            temp->next = newNode;
        }
    }

    printf("Enter value to insert at 3rd position: ");
    scanf("%d", &value);

    newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;

    temp = head;

    for (i = 1; i < 2; i++)
        temp = temp->next;

    newNode->next = temp->next;
    temp->next = newNode;

    printf("Linked List: ");

    temp = head;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }

    if (head->data == head->next->next->data)
        printf("\n1st and 3rd elements are same");
    else
        printf("\n1st and 3rd elements are not same");
```

```
    return 0;
}
```

Output:

```
Enter number of nodes: 4
Enter data: 10
Enter data: 20
Enter data: 30
Enter data: 40
Enter value to insert at 3rd position: 10
Linked List: 10 20 10 30 40
1st and 3rd elements are same
```

6. Implement two stacks in one array. Write routines to traverse the array and display the stacks.

Program:

```
#include <stdio.h>

#define MAX 10

int stack[MAX];
int top1 = -1;
int top2 = MAX;

void push1(int value) {
    if (top1 + 1 == top2) {
        printf("Stack Overflow\n");
        return;
    }

    stack[++top1] = value;
}

void push2(int value) {
    if (top1 + 1 == top2) {
        printf("Stack Overflow\n");
        return;
    }

    stack[--top2] = value;
}

void pop1() {
    if (top1 == -1)
        printf("Stack 1 Underflow\n");
    else
        printf("Popped from Stack 1: %d\n", stack[top1--]);
}

void pop2() {
    if (top2 == MAX)
        printf("Stack 2 Underflow\n");
    else
        printf("Popped from Stack 2: %d\n", stack[top2++]);
}

void display() {
    int i;

    printf("Stack 1: ");
    for (i = 0; i <= top1; i++)
        printf("%d ", stack[i]);

    printf("\nStack 2: ");
    for (i = MAX - 1; i >= top2; i--)
        printf("%d ", stack[i]);

    printf("\n");
}

int main() {
    push1(10);
    push1(20);
    push1(30);

    push2(100);
    push2(200);
```

```
    push2(300);

    display();

    pop1();
    pop2();

    display();

    return 0;
}
```

Output:

```
Stack 1: 10 20 30
Stack 2: 300 200 100
Popped from Stack 1: 30
Popped from Stack 2: 300
Stack 1: 10 20
Stack 2: 200 100
```

7. Given a linked list containing n head nodes where every node in the linked list contains two pointers, flatten the list so that all the nodes are in ascending order.

Program:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
    struct Node *down;
};

struct Node* createNode(int data) {
    struct Node *newNode =
        (struct Node*)malloc(sizeof(struct Node));

    newNode->data = data;
    newNode->next = NULL;
    newNode->down = NULL;

    return newNode;
}

struct Node* merge(struct Node *a, struct Node *b) {
    struct Node dummy;
    struct Node *temp = &dummy;

    dummy.down = NULL;

    while (a != NULL && b != NULL) {
        if (a->data < b->data) {
            temp->down = a;
            a = a->down;
        } else {
            temp->down = b;
            b = b->down;
        }

        temp = temp->down;
    }

    if (a != NULL)
        temp->down = a;
    else
        temp->down = b;

    return dummy.down;
}

void display(struct Node *head) {
    while (head != NULL) {
        printf("%d ", head->data);
        head = head->down;
    }
}

int main() {
    struct Node *list1, *list2, *list3, *result;

    list1 = createNode(5);
    list1->down = createNode(7);
    list1->down->down = createNode(8);

    list2 = createNode(10);
```

```
    list2->down = createNode(17);
    list2->down->down = createNode(20);

    list3 = createNode(22);
    list3->down = createNode(28);
    list3->down->down = createNode(40);
    list3->down->down->down = createNode(45);

    result = merge(list1, list2);
    result = merge(result, list3);

    printf("Flattened sorted list:\n");
    display(result);

    return 0;
}
```

Output:

```
Flattened sorted list:
5 7 8 10 17 20 22 28 40 45
```

8. Given K sorted linked lists, merge them into a single list while maintaining their sorted order.

Program:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node* createNode(int data) {
    struct Node *newNode =
        (struct Node*)malloc(sizeof(struct Node));

    newNode->data = data;
    newNode->next = NULL;

    return newNode;
}

struct Node* merge(struct Node *a, struct Node *b) {
    struct Node dummy;
    struct Node *temp = &dummy;

    dummy.next = NULL;

    while (a != NULL && b != NULL) {
        if (a->data <= b->data) {
            temp->next = a;
            a = a->next;
        } else {
            temp->next = b;
            b = b->next;
        }

        temp = temp->next;
    }

    if (a != NULL)
        temp->next = a;
    else
        temp->next = b;

    return dummy.next;
}

void display(struct Node *head) {
    while (head != NULL) {
        printf("%d ", head->data);
        head = head->next;
    }
}

int main() {
    struct Node *l1, *l2, *l3, *result;

    l1 = createNode(1);
    l1->next = createNode(4);
    l1->next->next = createNode(7);

    l2 = createNode(2);
    l2->next = createNode(5);
    l2->next->next = createNode(8);
```

```
    l3 = createNode(3);
    l3->next = createNode(6);
    l3->next->next = createNode(9);

    result = merge(l1, l2);
    result = merge(result, l3);

    printf("Merged Linked List:\n");
    display(result);

    return 0;
}
```

Output:

```
Merged Linked List:
1 2 3 4 5 6 7 8 9
```

9. Swap the kth node from the beginning with the kth node from the end in a linked list.

Program:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

int main() {
    struct Node *head = NULL, *newNode, *temp;
    struct Node *first, *second;
    int n, k, i, j;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        newNode = (struct Node*)malloc(sizeof(struct Node));

        printf("Enter data: ");
        scanf("%d", &newNode->data);

        newNode->next = NULL;

        if (head == NULL)
            head = newNode;
        else {
            temp = head;

            while (temp->next != NULL)
                temp = temp->next;

            temp->next = newNode;
        }
    }

    printf("Enter k: ");
    scanf("%d", &k);

    if (k > n) {
        printf("Invalid k");
        return 0;
    }

    first = head;

    for (i = 1; i < k; i++)
        first = first->next;

    second = head;

    for (i = 1; i < n - k + 1; i++)
        second = second->next;

    j = first->data;
    first->data = second->data;
    second->data = j;

    printf("After swapping:\n");

    temp = head;
```

```
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }

    return 0;
}
```

Output:

```
Enter number of nodes: 6
Enter data: 10
Enter data: 20
Enter data: 30
Enter data: 40
Enter data: 50
Enter data: 60
Enter k: 2
After swapping:
10 50 30 40 20 60
```

10. Multiply two numbers represented by two linked lists.

Program:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node* createNode(int data) {
    struct Node *newNode =
        (struct Node*)malloc(sizeof(struct Node));

    newNode->data = data;
    newNode->next = NULL;

    return newNode;
}

void insert(struct Node **head, int data) {
    struct Node *newNode = createNode(data);
    struct Node *temp;

    if (*head == NULL) {
        *head = newNode;
        return;
    }

    temp = *head;

    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
}

long long convertToNumber(struct Node *head) {
    long long num = 0;

    while (head != NULL) {
        num = num * 10 + head->data;
        head = head->next;
    }

    return num;
}

void display(struct Node *head) {
    while (head != NULL) {
        printf("%d", head->data);
        head = head->next;
    }

    printf("\n");
}

int main() {
    struct Node *num1 = NULL;
    struct Node *num2 = NULL;
    long long n1, n2, result;

    insert(&num1, 1);
    insert(&num1, 2);
```

```
    insert(&num1, 3);

    insert(&num2, 4);
    insert(&num2, 5);

    printf("First number: ");
    display(num1);

    printf("Second number: ");
    display(num2);

    n1 = convertToNumber(num1);
    n2 = convertToNumber(num2);

    result = n1 * n2;

    printf("Product = %lld\n", result);

    return 0;
}
```

Output:

```
First number: 123
Second number: 45
Product = 5535
```